

# 為雲端到雲端整合啟用本機執行作業 | Cloud-to-cloud | Google Home Developers

來源：<https://developers.home.google.com/codelabs/smarthome-local?hl=zh-tw>

步驟數：9

瞭解如何透過 Local Home SDK 啟用本地交車功能，藉此提高 Google 助理的智慧住宅整合成效。

---

# 目錄

1. [1. 事前準備](#)
2. [2. 開始使用](#)
3. [3. 執行範例應用程式](#)
4. [4. 更新雲端履行狀態](#)
5. [5. 設定店內取貨功能](#)
6. [6. 實作店面取貨功能](#)
7. [7. 啟動智慧型洗衣機](#)
8. [8. 對 TypeScript 應用程式進行偵錯](#)
9. [9. 恭喜](#)

步驟 1 · 約 4 分鐘

為雲端整合啟用本機出貨

## 程式碼研究室簡介

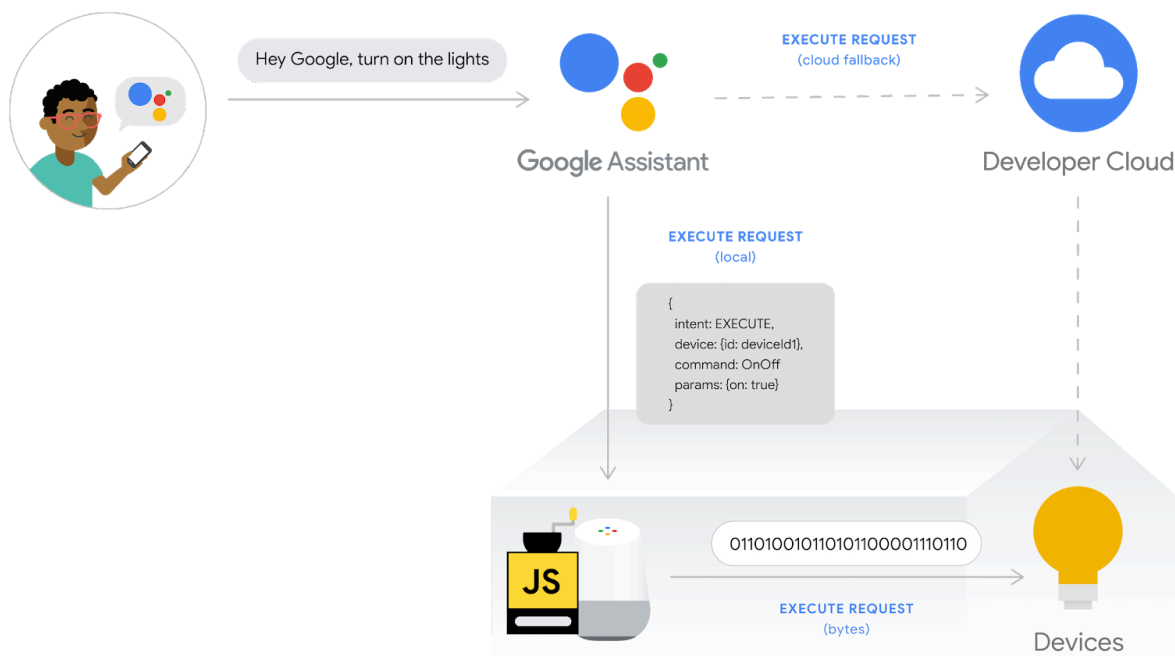
subject上次更新時間：5月 12, 2026

account\_circle作者：Google 員工

### 1. 事前準備

智慧住宅整合功能可讓 Google 助理控制使用者住家中的連結裝置。如要建構雲端對雲端整合，您必須提供可處理[智慧住宅意圖](#)的雲端 Webhook 端點。舉例來說，當使用者說出「Ok Google，開燈」時，Google 助理會將指令傳送至雲端服務，更新裝置狀態。

Local Home SDK 會新增本機路徑，將智慧型住宅意圖直接轉送至 Google Home 裝置，藉此提升智慧型住宅整合程度，並在處理使用者指令時提高穩定性及縮短延遲時間。您可以使用 TypeScript 或 JavaScript 編寫及部署本機履行應用程式，在任何 Google Home 智慧音箱或 Google Nest 智慧螢幕上識別裝置並執行指令。然後，應用程式會使用現有的標準通訊協定，透過區域網路直接與使用者的現有智慧型裝置通訊，以執行指令。



## 必要條件

- [建立雲端對雲端整合開發人員指南](#)
- [智慧型住宅洗衣機](#)程式碼研究室
- [店面取貨](#)開發人員指南

## 建構項目

在本程式碼研究室中，您將部署先前使用 Firebase 建構的智慧住宅整合功能，然後在[開發人員控制台](#)中套用掃描設定，並使用 TypeScript 建構本機應用程式，將以 Node.js 編寫的指令傳送至虛擬洗衣機裝置。

## 課程內容

- 如何在 Play 管理中心啟用及設定店內取貨功能。
- 如何使用 Local Home SDK 編寫本機執行應用程式。
- 如何對 Google Home 音箱或 Google Nest 智慧螢幕上載入的本機服務應用程式進行偵錯。

## 軟硬體需求

- 最新版 [Google Chrome](#)
- 已安裝 Google Home 應用程式的 iOS 或 Android 裝置
- Google Home 智慧音箱或 Google Nest 智慧螢幕
- [Node.js](#) 24 以上版本
- [Google 帳戶](#)
- Google Cloud [帳單帳戶](#)
- [Firebase](#) 的 Blaze (即付即用) 方案。

注意：GitHub 存放區中有一些依附元件未列於此處。您必須遵循 GitHub 存放區 `package.json` 檔案中指定的依附元件版本。

---

## 2. 開始使用

### 啟用活動控制項

如要使用 Google 助理，您必須與 Google 分享特定活動記錄。Google 助理需要這項資料才能正常運作，但分享資料的要求並非 SDK 專屬。如要分享這項資料，請建立 [Google 帳戶](#) (如果還沒有帳戶)。您可以使用任何 Google 帳戶，不一定要是開發人員帳戶。

開啟要與 Google 助理搭配使用的 Google 帳戶的[活動控制項頁面](#)。

確認已啟用下列切換開關：

- **網路和應用程式活動** - 此外，請務必勾選「包括 Chrome 歷史記錄，以及採用 Google 服務的網站、應用程式和裝置上的活動記錄」核取方塊。
- **裝置資訊**
- **語音和音訊活動**

手機和 Google Home 開發人員控制台必須使用相同的 Google 帳戶，才能完整執行這個專案。

### 建立雲端對雲端整合專案

1. 前往[開發人員控制台](#)。
2. 按一下「建立專案」，輸入專案名稱，然後按一下「建立專案」。

# Create a project

Create a project to get started.

Project name

Project name\*

30 characters max

Create project

Already have a Works with Assistant, cloud, or Local Home SDK project? [Import existing project](#).

## 選取雲端對雲端整合

在開發人員控制台的「專案首頁」中，選取「Cloud-to-cloud」下方的「新增雲端對雲端整合」。

### Cloud-to-cloud

Cloud-connected devices that support smart home Actions. Devices can be controlled through the Google Home app and Google Assistant. [Learn more](#)

[+ Add cloud-to-cloud integration](#)

No cloud-to-cloud integrations yet.

這個專案使用 [Cloud Functions for Firebase](#)，因此您必須將帳單帳戶與專案建立關聯。Actions 專案預設不會建立帳單帳戶。詳情請參閱「[建立新的帳單帳戶](#)」一文。

## 安裝 Firebase CLI

Firebase 指令列介面 (CLI) 可讓您在本地提供網頁應用程式，並將網頁應用程式部署至 Firebase 託管。

如要安裝 CLI，您必須先安裝 [npm](#)，這通常會隨附於 [Node.js](#)。

如要安裝 CLI，請從終端機執行下列 npm 指令：

```
npm install -g firebase-tools
```

如果無法正常運作，建議您變更 npm 權限。

如要確認 CLI 是否已正確安裝，請執行下列指令：

```
firebase --version
```

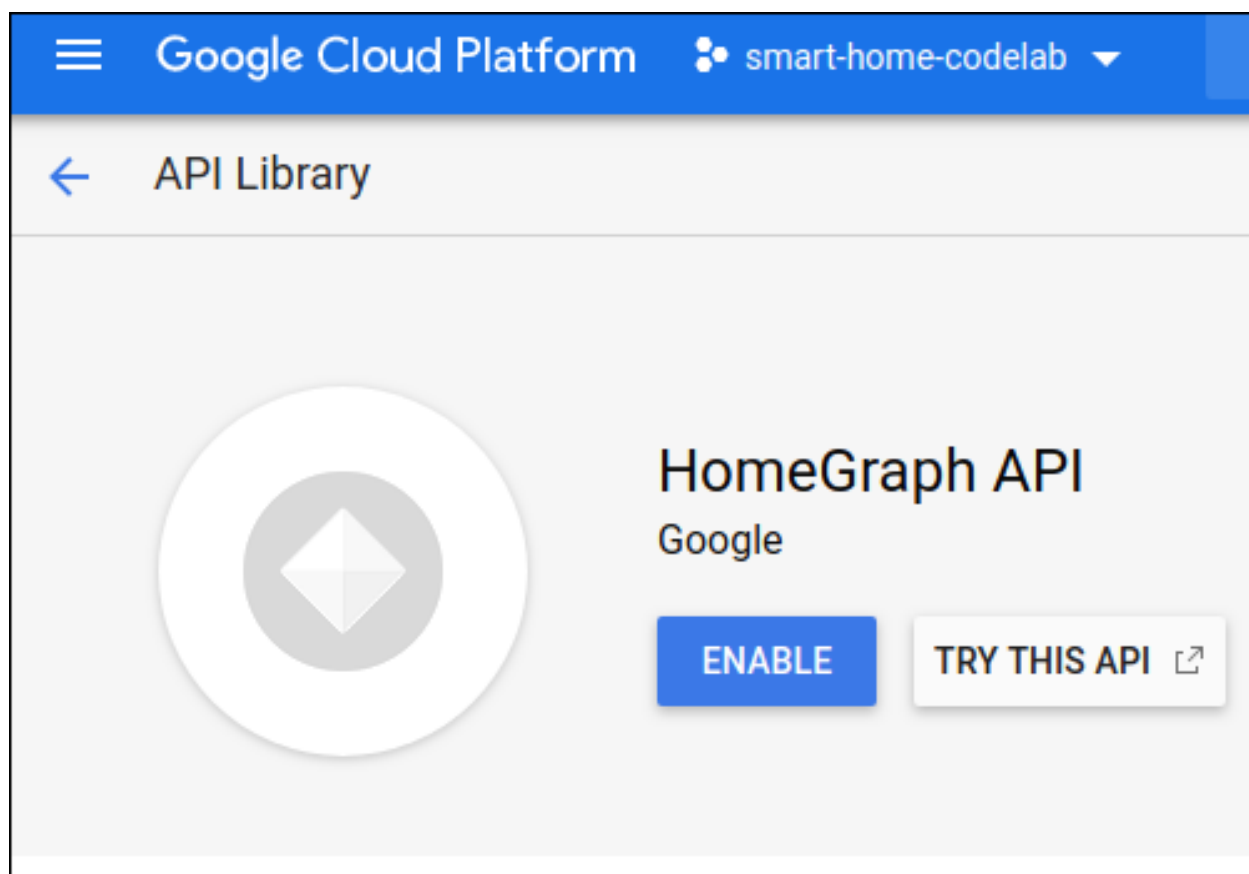
執行下列指令，透過 Google 帳戶授權 Firebase CLI：

```
firebase login
```

## 啟用 HomeGraph API

HomeGraph API 可讓您在使用者 Home Graph 中儲存及查詢裝置和裝置狀態。如要使用這項 API，請先開啟 Google Cloud 控制台並啟用 HomeGraph API。

在 Google Cloud 控制台中，請務必選取與整合項目相符的專案 `<project-id>`。接著，在 HomeGraph API 的「API 程式庫」畫面中，按一下「啟用」。



### 3. 執行範例應用程式

現在您已設定開發環境，可以部署入門專案，確認一切設定正確無誤。

## 取得原始碼

點選下方連結，即可在開發機器上為本程式碼研究室下載範例：

...或者，您也可以從指令列複製 GitHub 存放區：

```
git clone https://github.com/google-home/smarthome-local.git
```

注意：程式碼研究室存放區包含三個目錄：

- `app-start` — 您將以此為基礎建構的起始程式碼
- `app-done`：完成的店內取貨應用程式完整程式碼
- `virtual-device`：以 Node.js 編寫的模擬洗衣機裝置

如要執行完成的店內取貨應用程式，請在[開發人員控制台](#)中建立專案。

## 關於專案

入門專案包含下列子目錄：

- `public` — 前端網頁 UI，可控制及監控智慧洗衣機
- `functions`：實作雲端履單的 Cloud 函式，用於雲端整合
- `local` — 骨架店面出貨應用程式專案，其中意圖處理常式已存根化 `index.ts`

提供的雲端履行服務包含 `index.js` 中的下列函式：

- `fakeauth`：帳戶連結的授權端點
- `faketoken` - 帳戶連結的權杖端點
- `smarthome` - 智慧型住宅意圖執行要求端點
- `reportstate`：在裝置狀態變更時呼叫 HomeGraph API
- `updateDevice`：虛擬裝置用來觸發「回報狀態」的端點

提示：尋找 `TODO` 標記

範例專案會提供提示，指出需要變更或新增程式碼的位置。例如：

```
// TODO: Implement local fulfillment
```



## 連結至 Firebase

前往 `app-start` 目錄，然後使用 Cloud-to-Cloud 整合專案設定 Firebase CLI：

```
cd app-start
firebase use <project-id>
```

提示：如果您不知道 Cloud-to-Cloud 整合的專案 ID，可以在開發人員控制台的「專案首頁」下方找到「專案詳細資料」。

## 設定 Firebase 專案

初始化 Firebase 專案。

```
firebase init
```

選取 CLI 功能、「即時資料庫」和「函式」功能。

```
? Which Firebase features do you want to set up for this directory? Press Space to
select features, then Enter
to confirm your choices. (Press <space> to select, <a> to toggle all, <i> to invert
selection, and <enter> to
proceed)
>( ) Data Connect: Set up a Firebase Data Connect service
( ) Firestore: Configure security rules and indexes files for Firestore
( ) Genkit: Setup a new Genkit project with Firebase
(*) Functions: Configure a Cloud Functions directory and its files
( ) App Hosting: Configure an apphosting.yaml file for App Hosting
( ) Hosting: Configure files for Firebase Hosting and (optionally) set up GitHub
Action deploys
( ) Storage: Configure a security rules file for Cloud Storage
( ) Emulators: Set up local emulators for Firebase products
( ) Remote Config: Configure a template file for Remote Config
( ) Extensions: Set up an empty Extensions manifest
(*) Realtime Database: Configure a security rules file for Realtime Database and
(optionally) provision
default instance
( ) Data Connect: Set up a Firebase Data Connect service
( ) Firestore: Configure security rules and indexes files for Firestore
```

這會為專案初始化必要的 API 和功能。

系統提示時，請初始化即時資料庫。您可以沿用資料庫執行個體的預設位置。

? It seems like you haven't initialized Realtime Database in your project yet. Do you want to set it up?

Yes

? Please choose the location for your default Realtime Database instance:

us-central1

由於您使用的是入門專案程式碼，請選擇安全性規則的預設檔案，並確保不會覆寫現有的資料庫規則檔案。

? File database.rules.json already exists. Do you want to overwrite it with the Realtime Database Security Rules for <project-ID>-default-rtdb from the Firebase Console?

No

如果重新初始化專案，系統詢問是否要初始化或覆寫程式碼集時，請選取「覆寫」。

? Would you like to initialize a new codebase, or overwrite an existing one?

Overwrite

設定函式時，請使用預設檔案，並確保不會覆寫專案範例中現有的 **index.js** 和 **package.json** 檔案。

? What language would you like to use to write Cloud Functions?

JavaScript

? Do you want to use ESLint to catch probable bugs and enforce style?

No

? File functions/package.json already exists. Overwrite?

No

? File functions/index.js already exists. Overwrite?

No

如果重新初始化專案，系統詢問是否要初始化或覆寫 functions/.gitignore 時，請選取「No」。

? File functions/.gitignore already exists. Overwrite?

No

? Do you want to install dependencies with npm now?

Yes

如果不小心啟用了 ESLint，可以透過以下兩種方法停用：

1. 使用 GUI 前往專案底下的 `../functions` 資料夾，選取隱藏檔案 `.eslintrc.js` 並刪除。請勿與名稱相似的 `.eslintrc.json` 混淆。

2. 使用指令列：

```
cd functions
rm .eslintrc.js
```

為確保 Firebase 設定正確無誤，請將 `washer-done` 目錄中的 `firebase.json` 檔案複製到 `washer-start` 目錄，覆寫 `washer-start` 中的檔案。

在 `washer-start` 目錄中：

```
cp -vp ../washer-done/firebase.json .
```

## 部署至 Firebase

安裝依附元件並設定專案後，即可首次執行應用程式。

```
firebase deploy
```

您應該會看到下列控制台輸出內容：

...

✓ Deploy complete!

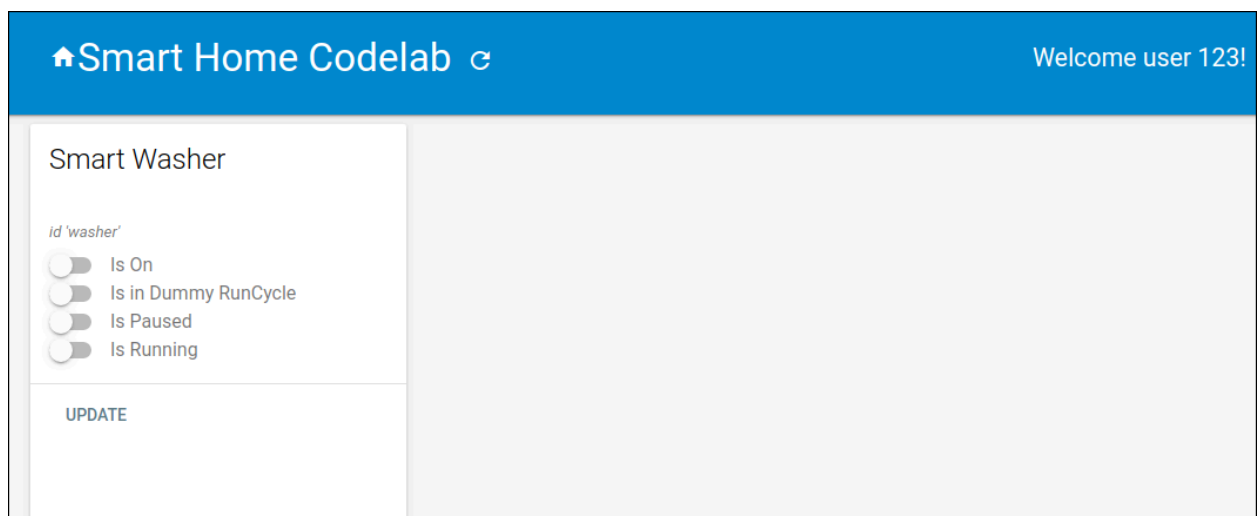
Project Console: <https://console.firebase.google.com/project/<project-id>/overview>

Hosting URL: <https://<project-id>.web.app>

這個指令會部署網頁應用程式，以及多個 [Cloud Functions for Firebase](#)。

**注意：**如果尚未為專案設定帳單，您必須升級至 **Blaze (即付即用)** 方案，[Firebase 函式](#)才能完成專案部署。

在瀏覽器中開啟「Hosting URL」(主機網址) (<https://<project-id>.web.app>)，即可查看網頁應用程式。您會看到下列介面：



這個網頁版 UI 代表第三方平台，可查看或修改裝置狀態。如要開始將裝置資訊填入資料庫，請按一下「更新」。頁面上不會顯示任何變更，但洗衣機的目前狀態會儲存在資料庫中。

Crashlytics, Performance, Test Lab

ANALYTICS  
Dashboard, Events, Audiences, Attrib...

GROW  
Predictions, Notifications, Remote C...

OnOff

...on: true

RunCycle + ×  
...dummy: false

StartStop

...isPaused: false

...isRunning: false

現在請使用[開發人員控制台](#)，將部署的雲端服務連結至 Google 助理。

## 設定開發人員控制台專案

在「開發」分頁中，為互動新增「顯示名稱」。這個名稱會顯示在 Google Home 應用程式中。

Project home

Matter

Cloud-to-cloud

Resources

Develop

Test

Certify

Launch

### Setup & configuration

Set up account linking and authentication, permissions, logs, and more.

Integration name \*

Codelab Washer

Device type \*

Washer

在「應用程式品牌宣傳」下方，上傳應用程式圖示的 `png` 檔案，大小為 144 x 144 像素，並命名為 `.png`。

### App branding

Your service name (the integration name above) and app icon will appear in the Google Home app.

App icon \*

File name must be the project ID (e.g., projectid.png). [View icon requirements](#)



File type: PNG transparent

File size: 144 x 144px

Upload image

如要啟用「帳戶連結」，請使用下列帳戶連結設定：

|        |                                                               |
|--------|---------------------------------------------------------------|
| 用戶端 ID | ABC123                                                        |
| 用戶端密鑰  | DEF456                                                        |
| 授權網址   | https://us-central1-<project-id>.cloudfunctions.net/fakeauth  |
| 權杖網址   | https://us-central1-<project-id>.cloudfunctions.net/faketoken |

請務必將網址更新為您的專案 ID。

## Account Linking

Authentication allows you to link your users' Google Accounts with their accounts in your authentication system. Then you can identify your users when your fulfillment receives a smart home intent.



[Start here](#). Read the OAuth 2.0 account linking guide before continuing.

### OAuth Client ID \*

String that identifies the request origin as Google.

Client ID\*

ABC123

### Client secret \*

Secret string that you registered with Google for your Client ID.

Client secret\*

.....

### Authorization URL \*

Endpoint for your sign-in web page that supports OAuth 2.0 code or implicit flows.

Authorization URL\*

`https://us-central1-<project-id>.cloudfunctions.net/fakeauth`

Please enter a valid https link

### Token URL \*

OAuth 2.0 endpoint for token exchange.

Token URL\*

`https://us-central1-<project-id>.cloudfunctions.net/faketoken`

Please enter a valid https link

在「Cloud 實現網址」下方，輸入提供智慧住宅意圖實現功能的 Cloud 函式網址。

`https://us-central1-<project-id>.cloudfunctions.net/smarthome`

## Cloud fulfillment URL

Enter the fulfillment URL used to process your smart home intents.

Fulfillment URL \*

`https://us-central1-<project-id>.cloudfunctions.net/smarthome`

Please enter a valid https link

按一下「儲存」儲存專案設定，然後按一下「下一步：測試」，在專案中啟用測試。

Developer Console

smarthome-washer-dc

Project home

Matter

Cloud-to-cloud

Resources

Develop

Test

Certify

Launch

Analytics

### Develop

Edit the integration.

| Name           | Status | Last modified ↑       |
|----------------|--------|-----------------------|
| Codelab Washer | Draft  | 5:43 PM - Oct 2, 2024 |

Edit

Next: Test

#### Related content

##### Get started

Learn how to properly set up your environment to ensure your implementation will be a success.

[Learn more](#)

##### Supported devices

See which devices and traits are supported across apps and services.

[Learn more](#)

##### Codelabs

Get hands-on, step-by-step guidance with our technical product tutorials.

[Go to codelab](#)

現在可以開始實作必要的 Webhook，將裝置狀態與 Google 助理連結。

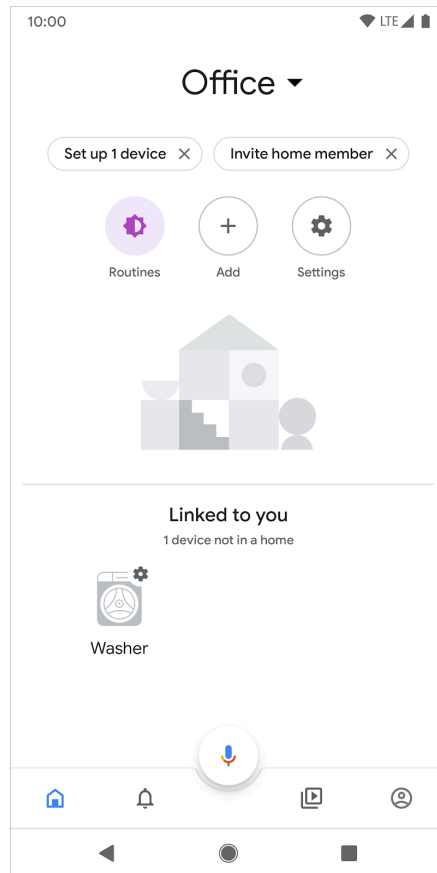
## 連結至 Google 助理

如要測試雲端對雲端整合，您必須將專案連結至 Google 帳戶。這樣一來，您就能透過登入相同帳戶的 Google 助理介面和 Google Home 應用程式進行測試。

**重要事項：**本程式碼研究室包含帳戶連結的實作項目，但不會實際檢查使用者憑證。在實際工作環境中，您應實作 OAuth 2.0 通訊協定，確保裝置安全無虞。

1. 在手機上開啟 Google 助理設定。請注意，您登入的帳戶應與控制台的帳戶相同。
2. 依序前往「Google 助理」>「設定」>「居家控制」（位於「Google 助理」下方）。
3. 按一下右上方的搜尋圖示。
4. 使用 **[test]** 前置字元搜尋測試應用程式，找出特定測試應用程式。
5. 選取該項目。Google 助理接著會向服務進行驗證，並傳送 **sync** 要求，請服務提供使用者的裝置清單。

開啟 Google Home 應用程式，確認是否能看到洗衣機裝置。



**注意：**如果在連結過程中遇到問題，可以使用 [Stackdriver](#) 查看錯誤記錄。

**重要事項：**如要使用本地出貨功能，你必須在同一個[住家圖表](#)結構中設定 Google Home 裝置和洗衣機。如果你在 Google Home 應用程式中設定多個住家結構，請務必將兩部裝置新增至同一個住家。

確認你可以在 Google Home 應用程式中使用語音指令控制洗衣機。此外，你應該也會在雲端服務的網頁前端 UI 中看到裝置狀態變更。

現在你可以開始在整合服務中新增店內取貨功能。



步驟 4 · 約 5 分鐘

## 4. 更新雲端履行狀態

如要支援本機出貨，您需要在雲端 SYNC 回應中新增名為 `otherDeviceIds` 的裝置專屬欄位，其中包含裝置的專屬本機 ID。這個欄位也會指出是否能在本機控制該裝置。

在 SYNC 回應中新增 `otherDeviceIds` 欄位，如以下程式碼片段所示：

### functions/index.js

```
app.onSync((body) => {
  return {
    requestId: body.requestId,
    payload: {
      agentUserId: '123',
      devices: [{
        id: 'washer',
        type: 'action.devices.types.WASHER',
        traits: [ ... ],
        name: { ... },
        deviceInfo: { ... },
        willReportState: true,
        attributes: {
          pausable: true,
        },
        otherDeviceIds: [{
          deviceId: 'deviceid123',
        }],
      }],
    },
  };
});
```

將更新後的專案部署至 Firebase：

```
firebase deploy --only functions
```

部署完成後，前往網頁使用者介面，然後按一下工具列中的「重新整理」



按鈕。這會觸發「要求同步」作業，讓 Google 助理接收更新後的 SYNC 回應資料。

Smart Home Codelab 

注意：您可以在 Firebase 控制台記錄中，確認同步要求是否成功執行。

|                |                                                                                   |             |                           |
|----------------|-----------------------------------------------------------------------------------|-------------|---------------------------|
| 3:26:17.849 PM |  | requestsync | Request SYNC for user 123 |
| 3:26:18.380 PM |  | requestsync | Request sync completed    |

步驟 5 · 約 5 分鐘

## 5. 設定店內取貨功能

在本節中，您將為雲端整合加入本機出貨的必要設定選項。開發期間，您會將本機執行要求應用程式發布至 Firebase 託管，Google Home 裝置可存取並下載該應用程式。

在 [Google Home 開發人員控制台](#) 中，前往畫面左側的「專案」>「雲端對雲端」，然後選取整合的「編輯」。在「設定與配置」頁面中，捲動至「店內取貨」，然後開啟這項設定。在每個測試網址欄位中輸入下列網址，插入專案 ID，然後按一下「儲存」：

```
https://<project-id>.web.app/local-home/index.html
```

## Local fulfillment



Lower latency and improve reliability by allowing your app to communicate directly with your devices to fulfill user commands.

### Testing URLs

Add your browser and Node.js compatible versions of your Local Home app for use during development.

Testing URL for Chrome

<https://project-id.firebaseio.com/local-home/index.html>

Testing URL for Node

<https://project-id.firebaseio.com/local-home/index.html>

### JavaScript files \*

To enable local fulfillment, upload the following files.

Max file size 8MB

[Upload JavaScript targeting Node](#)

Max file size 8MB

[Upload JavaScript targeting Chrome](#)

☐ Support local queries

### Device discovery \*

Enter scan configuration info so Google can discover your device on the local network.

[+ Add scan configuration](#)

注意：您會在下一節中，於相同網址發布本機住家應用程式。

接下來，我們需要定義 Google Home 裝置應如何[探索](#)本機智慧型裝置。本機住家平台支援多種裝置探索通訊協定，包括 mDNS、UPnP 和 UDP 廣播。您將使用 UDP 廣播來探索智慧洗衣機。

按一下「裝置探索」下方的「+ 新增掃描設定」，即可新增掃描設定。選取 UDP 做為通訊協定，然後填寫下列屬性：

| 欄位    | 說明                        | 建議值                                |
|-------|---------------------------|------------------------------------|
| 探索地址  | UDP 探索位址                  | 255.255.255.255                    |
| 廣播通訊埠 | Google Home 傳送 UDP 廣播的通訊埠 | 3311                               |
| 監聽通訊埠 | Google Home 監聽回應的通訊埠      | 3312                               |
| 探索封包  | UDP 廣播資料酬載                | 48656c6c6f4c6f63616c486f6d6553444b |

## Device discovery \*

Enter scan configuration info so Google can discover your device on the local network.

Discovery protocol \*  
UDP 

Discovery address \*  
255.255.255.255

Discovery packet \*  
48656c6c6f4c6f63616c486f6d6553444b

Listening port number \*  
3312

Broadcasting port number \*  
3311

+ Add field

+ Add scan configuration

提示：值 `48656c6c6f4c6f63616c486f6d6553444b` 是 `HelloLocalHomeSDK` 字串的十六進位編碼版本。

最後，按一下視窗底部的「儲存」，即可發布變更。

注意：開發人員控制台最多可能需要 20 分鐘，才會將設定變更傳播至 Google Home 裝置。

## 6. 實作店面取貨功能

您將使用 Local Home SDK [型別套件](#)，以 TypeScript 開發本機執行應用程式。查看範例專案中提供的架構：

### local/index.ts

```
/// <reference types="@google/local-home-sdk" />

import App = smarthome.App;
import Constants = smarthome.Constants;
import DataFlow = smarthome.DataFlow;
import Execute = smarthome.Execute;
import Intents = smarthome.Intents;
import IntentFlow = smarthome.IntentFlow;

...

class LocalExecutionApp {

  constructor(private readonly app: App) { }

  identifyHandler(request: IntentFlow.IdentifyRequest):
    Promise<IntentFlow.IdentifyResponse> {
    // TODO: Implement device identification
  }

  executeHandler(request: IntentFlow.ExecuteRequest):
    Promise<IntentFlow.ExecuteResponse> {
    // TODO: Implement local fulfillment
  }

  ...
}

const localHomeSdk = new App('1.0.0');
const localApp = new LocalExecutionApp(localHomeSdk);
localHomeSdk
  .onIdentify(localApp.identifyHandler.bind(localApp))
  .onExecute(localApp.executeHandler.bind(localApp))
  .listen()
  .then(() => console.log('Ready'))
  .catch((e: Error) => console.error(e));
```

店內取貨的核心元件是 `smarthome.App` 類別。範例專案會附加 `IDENTIFY` 和 `EXECUTE` 意圖的處理常式，然後呼叫 `listen()` 方法，通知 Local Home SDK 應用程式已準備就緒。

### 新增 IDENTIFY 處理常式

當 Google Home 裝置根據開發人員控制台中提供的掃描設定，在區域網路上探索到未經驗證的裝置時，區域網路 SDK 會觸發 `IDENTIFY` 處理常式。

同時，當 Google 發現相符的裝置時，平台會使用掃描結果資料叫用 `identifyHandler`。在應用程式中，掃描作業會使用 UDP 廣播進行，提供給 `IDENTIFY` 處理常式的掃描資料則包含本機裝置傳送的回應酬載。

處理常式會傳回 `IdentifyResponse` 執行個體，其中包含本機裝置的專屬 ID。在 `identifyHandler` 方法中加入下列程式碼，處理來自本機裝置的 UDP 回應，並判斷適當的本機裝置 ID：

## local/index.ts

```
identifyHandler(request: IntentFlow.IdentifyRequest):
  Promise<IntentFlow.IdentifyResponse> {
  console.log("IDENTIFY intent: " + JSON.stringify(request, null, 2));

  const scanData = request.inputs[0].payload.device.udpScanData;
  if (!scanData) {
    const err = new IntentFlow.HandlerError(request.requestId,
      'invalid_request', 'Invalid scan data');
    return Promise.reject(err);
  }

  // In this codelab, the scan data contains only local device ID.
  const localDeviceId = Buffer.from(scanData.data, 'hex');

  const response: IntentFlow.IdentifyResponse = {
    intent: Intents.IDENTIFY,
    requestId: request.requestId,
    payload: {
      device: {
        id: 'washer',
        verificationId: localDeviceId.toString(),
      }
    }
  };
  console.log("IDENTIFY response: " + JSON.stringify(response, null, 2));

  return Promise.resolve(response);
}
```

請注意，`verificationId` 欄位必須與 `SYNC` 回應中的其中一個 `otherDeviceIds` 值相符，這會將裝置標示為可在使用者[住家圖表](#)中進行店面取貨。Google 找到相符裝置後，即視為已驗證，可進行店面取貨。

**注意：**如果裝置是中樞裝置，並連線至其他智慧型裝置，請參閱[本地端執行要求指南](#)中的「支援中樞裝置後方的裝置」一節。

## 新增 EXECUTE 處理常式

當支援本機執行的裝置收到指令時，Local Home SDK 會觸發 `EXECUTE` 處理常式。本機意圖的內容等同於傳送至雲端服務的 `EXECUTE` 意圖，因此本機處理意圖的邏輯與在雲端處理意圖的方式類似。

應用程式可以使用 TCP/UDP Socket 或 HTTP(S) 要求與本機裝置通訊。在本程式碼研究室中，HTTP 是用來控制虛擬裝置的通訊協定。通訊埠號碼在 `index.ts` 中定義為 `SERVER_PORT` 變數。



在 `executeHandler` 方法中加入下列程式碼，處理傳入的指令並透過 HTTP 將指令傳送至本機裝置：

[local/index.ts](#)

```

executeHandler(request: IntentFlow.ExecuteRequest):
    Promise<IntentFlow.ExecuteResponse> {
        console.log("EXECUTE intent: " + JSON.stringify(request, null, 2));

        const command = request.inputs[0].payload.commands[0];
        const execution = command.execution[0];
        const response = new Execute.Response.Builder()
            .setRequestId(request.requestId);

        const promises: Array<Promise<void>> = command.devices.map((device) => {
            console.log("Handling EXECUTE intent for device: " + JSON.stringify(device));

            // Convert execution params to a string for the local device
            const params = execution.params as IWasherParams;
            const payload = this.getDataForCommand(execution.command, params);

            // Create a command to send over the local network
            const radioCommand = new DataFlow.HttpRequestData();
            radioCommand.requestId = request.requestId;
            radioCommand.deviceId = device.id;
            radioCommand.data = JSON.stringify(payload);
            radioCommand.dataType = 'application/json';
            radioCommand.port = SERVER_PORT;
            radioCommand.method = Constants.HttpOperation.POST;
            radioCommand.isSecure = false;

            console.log("Sending request to the smart home device:", payload);

            return this.app.getDeviceManager()
                .send(radioCommand)
                .then(() => {
                    const state = {online: true};
                    response.setSuccessState(device.id, Object.assign(state, params));
                    console.log(`Command successfully sent to ${device.id}`);
                })
                .catch((e: IntentFlow.HandlerError) => {
                    e.errorCode = e.errorCode || 'invalid_request';
                    response.setErrorState(device.id, e.errorCode);
                    console.error('An error occurred sending the command', e.errorCode);
                });
        });

        return Promise.all(promises)
            .then(() => {
                return response.build();
            })
            .catch((e) => {
                const err = new IntentFlow.HandlerError(request.requestId,
                    'invalid_request', e.message);
                return Promise.reject(err);
            });
    }

```

```
});  
}
```

提示：[DeviceManager](#) 介面會在內部將裝置 ID 對應至透過區域網路與該裝置通訊所需的位址。

## 編譯 TypeScript 應用程式

前往 `local/` 目錄，然後執行下列指令，下載 TypeScript 編譯器並編譯應用程式：

```
cd local  
npm install  
npm run build
```

這會編譯 `index.ts` (TypeScript) 來源，並將下列內容放入 `public/local-home/` 目錄：

- `bundle.js`：編譯後的 JavaScript 輸出內容，包含本機應用程式和依附元件。
- `index.html`：用於放送應用程式的本機代管頁面，方便在裝置端進行測試。

注意：部署完成後，您可以在先前於[開發人員控制台](#)「設定本機住家 SDK」部分提供的測試網址，找到代管的 `index.html` 頁面。

## 部署測試專案

將更新後的專案檔案部署至 Firebase 託管，以便透過 Google Home 裝置存取這些檔案。

```
firebase deploy --only hosting
```

## 7. 啟動智慧型洗衣機

現在可以測試本機執行要求應用程式與智慧洗衣機之間的通訊！本程式碼研究室的入門專案包含以 Node.js 編寫的虛擬智慧洗衣機，可模擬使用者在本機控制的智慧洗衣機。

### 設定裝置

您必須設定虛擬裝置，使用您在開發人員控制台中，套用至裝置探索掃描設定的相同 UDP 參數。此外，您還需要告知虛擬裝置要回報哪個本機裝置 ID，以及裝置狀態變更時，雲端對雲端整合要用於「回報狀態」事件的專案 ID。

| 參數               | 建議值               |
|------------------|-------------------|
| deviceId         | deviceid123       |
| discoveryPortOut | 3311              |
| discoveryPacket  | HelloLocalHomeSDK |
| projectId        | 雲端對雲端整合的專案 ID     |

### 啟動裝置

前往 `virtual-device/` 目錄並執行裝置指令碼，將設定參數做為引數傳遞：

```
cd virtual-device
npm install
npm start -- \
  --deviceId=deviceid123 --projectId=<project-id> \
  --discoveryPortOut=3311 --discoveryPacket=HelloLocalHomeSDK
```

確認裝置指令碼是否使用預期參數執行：

```
(...): UDP Server listening on 3311
(...): Device listening on port 3388
(...): Report State successful
```

## 8. 對 TypeScript 應用程式進行偵錯

在下一節中，您將確認 Google Home 裝置是否能透過區域網路正確掃描、識別虛擬智慧洗衣機，並傳送指令。您可以使用 [Google Chrome 開發人員工具](#) 連線至 Google Home 裝置、查看控制台記錄，以及偵錯 TypeScript 應用程式。

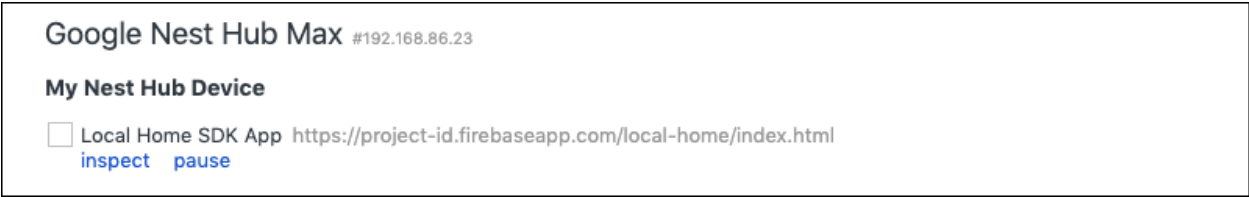
**注意：**開發機器和 Google Home 裝置必須連上相同的 Wi-Fi 網路，Chrome 開發人員工具才能在使用者介面中正確顯示裝置。

### 連線 Chrome 開發人員工具

如要將偵錯工具連線至本機履行應用程式，請按照下列步驟操作：

1. 確認你已將 Google Home 裝置連結至具備 [Developer Console](#) 專案存取權的使用者。
2. 重新啟動 Google Home 裝置，這樣裝置就能取得 HTML 的網址，以及您在開發人員控制台中輸入的掃描設定。
3. 在開發機器上啟動 Chrome。
4. 開啟新的 Chrome 分頁，然後在網址欄位中輸入 `chrome://inspect`，啟動檢查器。

網頁上應會顯示裝置清單，而應用程式網址則會顯示在 Google Home 裝置名稱下方。



Google Nest Hub Max #192.168.86.23

My Nest Hub Device

☐ Local Home SDK App <https://project-id.firebaseio.com/local-home/index.html>

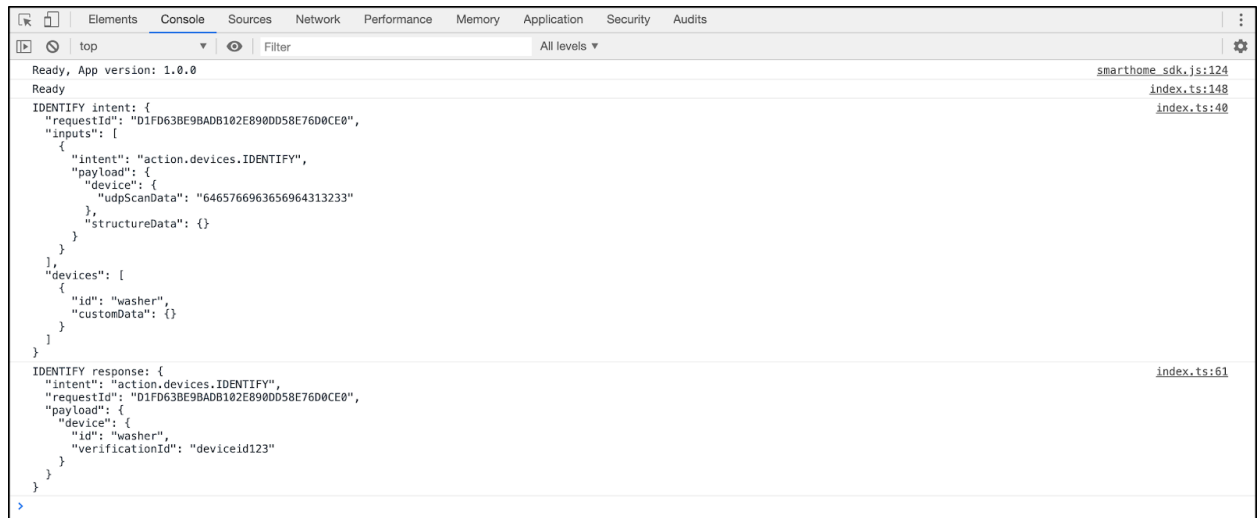
[inspect](#) [pause](#)

**重要事項：**如果沒有顯示網址，請取消連結並重新連結 Cloud-to-cloud 整合專案，然後重新啟動 Google Home 裝置。Play 管理中心的設定最多可能需要 20 分鐘才會生效。

### 啟動檢查器

按一下應用程式網址下方的「檢查」，啟動 Chrome 開發人員工具。選取「控制台」分頁，確認您可以看到 TypeScript 應用程式列印的 `IDENTIFY` 意圖內容。

**提示：**掃描時，Google Home 裝置會定期在網路上傳送探索封包。每次掃描結果都會觸發 `IDENTIFY` 意圖，直到應用程式驗證探索到的裝置 ID 為止。



```
Ready, App version: 1.0.0 smarthome_sdk.js:124
Ready index.ts:148
IDENTIFY intent: {
  "requestId": "D1FD63BE9BAD8182E890DD58E76D0CE0",
  "inputs": [
    {
      "intent": "action.devices.IDENTIFY",
      "payload": {
        "device": {
          "udpScanData": "6465766963656964313233"
        },
        "structureData": {}
      }
    }
  ],
  "devices": [
    {
      "id": "washer",
      "customData": {}
    }
  ]
}
IDENTIFY response: {
  "intent": "action.devices.IDENTIFY",
  "requestId": "D1FD63BE9BAD8182E890DD58E76D0CE0",
  "payload": {
    "device": {
      "id": "washer",
      "verificationId": "deviceid123"
    }
  }
}
```

這項輸出內容表示本機執行要求應用程式已成功探索及識別虛擬裝置。

## 測試本機履行

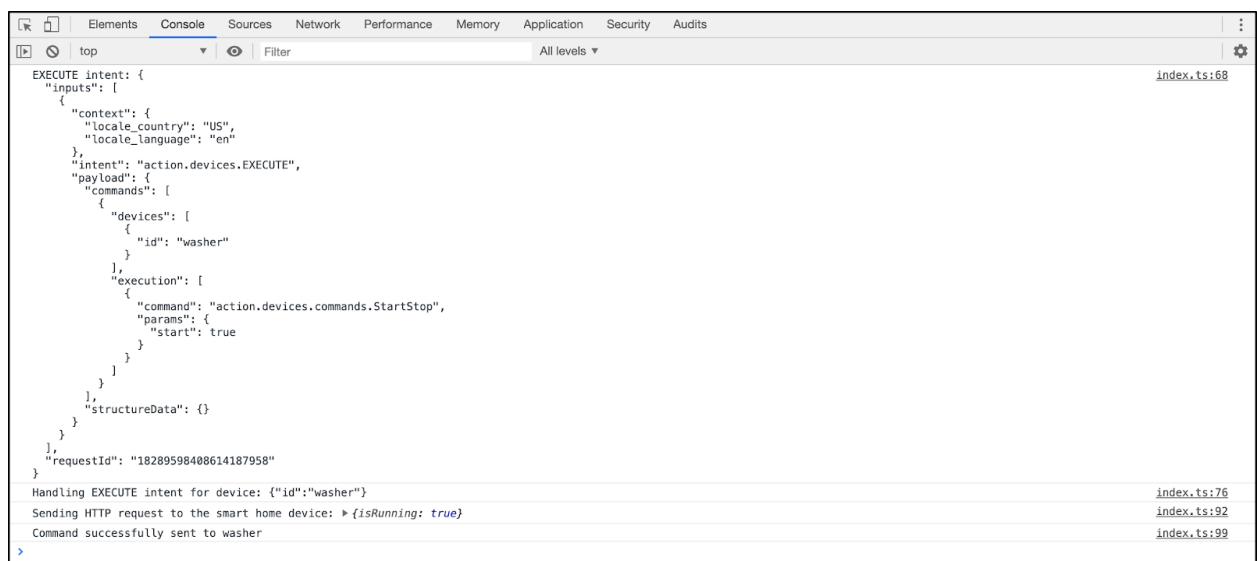
使用 Google Home 應用程式中的觸控手勢，或透過語音指令操作 Google Home 裝置，例如：

「Ok Google，開啟洗衣機。」

「Ok Google，啟動洗衣機。」

「Ok Google，停止洗衣機。」

這應該會觸發平台，將 **EXECUTE** 意圖傳送至 TypeScript 應用程式。



```
EXECUTE intent: {
  "inputs": [
    {
      "context": {
        "locale_country": "US",
        "locale_language": "en"
      },
      "intent": "action.devices.EXECUTE",
      "payload": {
        "commands": [
          {
            "devices": [
              {
                "id": "washer"
              }
            ],
            "execution": [
              {
                "command": "action.devices.commands.StartStop",
                "params": {
                  "start": true
                }
              }
            ]
          }
        ],
        "structureData": {}
      }
    }
  ],
  "requestId": "18289598408614187958"
}
Handling EXECUTE intent for device: {"id":"washer"} index.ts:76
Sending HTTP request to the smart home device: ▶ {isRunning: true} index.ts:92
Command successfully sent to washer index.ts:99
```

確認每次發出指令後，都能看到智慧洗衣機的狀態變化。

```
...
***** The washer is RUNNING *****
...
***** The washer is STOPPED *****
```

注意：由於裝置會實作「回報狀態」，因此您也應該會在前端網頁 UI 中看到裝置狀態變更。

### Smart Washer

*id 'washer'*

☒

Is On

☐

Is in Dummy RunCycle

☐

Is Paused

☒

Is Running

UPDATE

步驟 9 · 約 0 分鐘

## 9. 恭喜



恭喜！您已使用 Local Home SDK，將本機執行整合至雲端對雲端整合。

## 瞭解詳情

你也可以嘗試下列做法：

- 變更掃描設定，讓掃描作業正常運作。例如，嘗試使用其他 UDP 連接埠或探索封包。
  - 修改虛擬智慧型裝置的程式碼集，在嵌入式裝置 (例如 Raspberry Pi) 上執行，並使用 LED 或螢幕顯示目前狀態。
-